

Jon Spurring

Department of Computer Science,
University of Copenhagen

Learning to Program with F#

2025-08-20

Preface

This book has been written as an introduction to programming for novice programmers. It is used in the first programming course at the University of Copenhagen's bachelor in computer science program. It started as a few chapters in 2016 and was to a large extent completed in 2017 using Mono. In 2023 the examples were migrated to DotNet, and in 2024 it was released to the public domain on GitHub.

The book was developed alongside the course Programmering og Problemløsning (programming and problem solving) and I am very thankful for the positive feedback and suggestions numerous people have given me. I would particularly like to thank Malthe Sporning for his insightful and detailed comments to every (!) page of this book. I also would like to acknowledge the invaluable feedback from my co-teachers: Torben Mogensen, Martin Elsmann, Christina Lioma; my many teaching assistants during the early development of the book. And finally, thanks to all the students of our course who have had the patience and endurance to labor and enjoy learning to program using F#.

The book has been typeset in L^AT_EX, and all programs in this version have been run in DotNet version 9.0.301 under MacOS version 15.6. The output of most of the examples are direct result of running the relevant programs.

Jon Sporning
Professor, Ph.d.
Department of Computer Science,
University of Copenhagen
2025-08-20

Contents

1	Introduction	1
1.1	How to Learn to Solve Problems by Programming	1
1.2	How to Solve Problems	2
1.3	Approaches to Programming	3
1.4	Why Use F#	3
1.5	How to Read This Book	4
 Part I Getting started		
2	Solving problems by writing a program	7
2.1	Executing F# programs on a computer	9
2.2	Values have types and types reduce the risk of programming errors	10
2.3	Organizing often used code in functions	11
2.4	Asking the user for input	12
2.5	Conditionally execute code	13
2.6	Repeatedly execute code	14
2.7	Programming as a form of communication	16
2.8	Key Concepts and Terms in This Chapter	17
3	Using F# as a Calculator	19
3.1	Literals and Basic Types	21
3.2	Operators on Basic Types	25

<i>Contents</i>	viii
3.3 Boolean Arithmetic	28
3.4 Integer Arithmetic	29
3.5 Floating Point Arithmetic	31
3.6 Char and String Arithmetic	32
3.7 Programming Intermezzo: Hand Conversion Between Decimal and Binary Numbers	33
3.8 Key Concepts and Terms in This Chapter	34
4 Values, Functions, and Statements	37
4.1 Value Bindings	40
4.2 Function Bindings	42
4.3 Do-Bindings	47
4.4 Conditional Expressions	48
4.5 Tracing code by hand	51
4.6 Key Concepts and Terms in This Chapter	53
5 Programming with Types	55
5.1 Type Products: Tuples	56
5.2 Type Sums: Discriminated Unions	58
5.3 Records	59
5.4 Type Abbreviations	61
5.5 Variable Types	62
5.6 Key Concepts and Terms in This Chapter	64
6 Making Programs and Documenting Them	67
6.1 The 8-step Guide to Writing Functions	68
6.2 Programming as a Communication Activity	70
6.3 Key Concepts and Terms in This Chapter	72
Part II Declarative Programming Paradigms	
7 Lists	77

7.1	The List Module	82
7.2	Programming Intermezzo: Word Statistics	86
7.3	Key concepts and terms in this chapter	87
8	Recursion Revisited	89
8.1	Recursive Functions	91
8.2	The Call Stack and Tail Recursion	93
8.3	Mutually Recursive Functions	96
8.4	Recursive types	97
8.5	Tracing Recursive Programs	97
8.6	Key Concepts and Terms in This Chapter	99
9	Organising Code in Libraries and Application Programs	101
9.1	Dotnet projects: Libraries and applications	102
9.2	Libraries and applications	104
9.3	Specifying a Module's Interface with a Signature File	104
9.4	Programming Intermezzo: Postfix Arithmetic with a Stack	106
9.5	Generic Modules	109
9.6	Key Concepts and Terms in This Chapter	109
10	Higher-Order Functions	111
10.1	Functions as Values	112
10.2	The Function Composition Operator	113
10.3	Currying	113
10.4	Key concepts and terms in this chapter	115
11	Data Structures	117
11.1	Queues	118
11.2	Trees	119
11.3	Programming intermezzo: Sorting Integers with a Binary Tree	122
11.4	Sets	125

11.5 Maps	127
11.6 Key concepts and terms in this chapter	128

Part III Imperative Programming Paradigms

12 Working With Files	135
12.1 Command Line Arguments	136
12.2 Interacting With the Console	137
12.3 Exceptions	139
12.4 Storing and Retrieving Data From a File	145
12.5 Working With Files and Directories	148
12.6 Programming intermezzo: Name of Existing File Dialogue	150
12.7 Resource Management	151
12.8 Key concepts and terms in this chapter	152
13 Imperative programming	153
13.1 Variables	154
13.2 While and For Loops	157
13.3 Programming Intermezzo: Imperative Fibonacci numbers	159
13.4 Arrays	160
13.4.1 Array Properties and Methods	163
13.4.2 The Array Module	163
13.4.3 Multidimensional Arrays	164
13.4.4 The Array2D Module	167
13.5 Tracing Imperative Programs	168
13.5.1 Tracing Loops	168
13.5.2 Tracing Mutable Values	171
13.6 Key Concepts and Terms in This Chapter	172
14 Testing Programs	173
14.1 Black-box Testing	176

14.2 White-box Testing	177
14.3 Key Concepts and Terms in This Chapter	180
15 Classes and Objects	183
15.1 Constructors and Members	184
15.2 Accessors	186
15.3 Objects are Reference Types	189
15.4 Static Classes	189
15.5 Recursive Members and Classes	190
15.6 Function and Operator Overloading	191
15.7 Additional Constructors	193
15.8 Programming Intermezzo: Two Dimensional Vectors	195
15.9 Key Concepts and Terms in This Chapter	198
16 Derived Classes	199
16.1 Inheritance	200
16.2 Interfacing with the <code>printf</code> Family	203
16.3 Abstract Classes	204
16.4 Interfaces	205
16.5 Programming Intermezzo: Chess	207
16.6 Key Concepts and Terms in This Chapter	216
17 Object-Oriented Design	217
17.1 Identification of Objects, Behaviors, and Interactions by Nouns-and-Verbs	218
17.2 Class Diagrams in the Unified Modelling Language	219
17.2.1 Associations	220
17.2.2 Inheritance-type relations	222
17.2.3 Packages	224
17.3 Programming Intermezzo: Designing a Racing Game	224
17.4 Key Concepts and Terms in This Chapter	227

Part IV Appendices

A	The Console in Windows, MacOS X, and Linux	231
A.1	The Basics	231
A.2	Windows	232
A.3	MacOS X and Linux	235
B	Number Systems on the Computer	239
B.1	Binary Numbers	239
B.2	IEEE 754 Floating Point Standard	240
C	Commonly Used Character Sets	243
C.1	ASCII	243
C.2	ISO/IEC 8859	245
C.3	Unicode	245
D	Common Language Infrastructure	247
	References	249
	Index	251